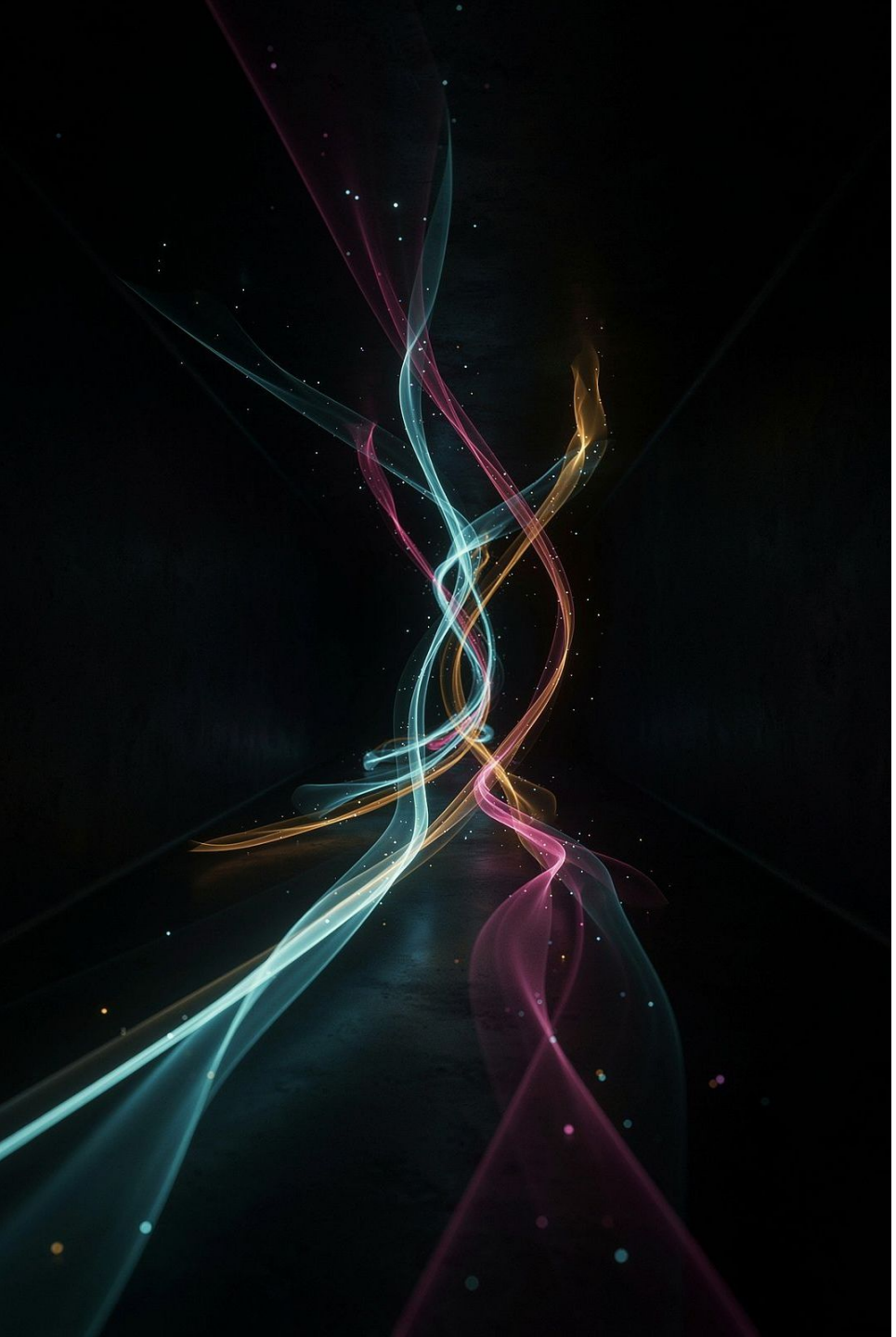


Sprint 12 · Lenguaje, Secuencias y Modelos Generativos

S12.1 · RNN, LSTM, GRU y Series Temporales

Un recorrido por las arquitecturas recurrentes y las técnicas fundamentales para modelar datos que evolucionan en el tiempo. Desde las redes RNN clásicas hasta LSTM y GRU, pasando por la validación temporal y la comparación con métodos estadísticos.

2 · Redes Recurrentes (RNN)

Una **red neuronal recurrente** (RNN) procesa las entradas una a una, manteniendo un **estado oculto** *h_t* que actúa como memoria a corto plazo. En cada paso de tiempo, la red combina la entrada actual *x_t* con el estado anterior *h_{t-1}* para producir el nuevo estado:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Este mecanismo permite, en teoría, que la red recuerde información de pasos anteriores. Sin embargo, en la práctica esta memoria se **desvanece rápidamente** con secuencias largas.

Idea clave

La RNN comparte los mismos pesos **W** en cada paso de tiempo. Esto la hace eficiente en parámetros, pero genera el problema del gradiente que veremos a continuación.

- Entrada: *x_t* en cada instante
- Estado oculto: *h_t* como memoria
- Salida: *y_t* opcional en cada paso

Vanishing y Exploding Gradient

Vanishing Gradient 📉

Durante la retropropagación a través del tiempo (**BPTT**), el gradiente se multiplica repetidamente por la matriz de pesos W_h. Si sus valores son menores que 1, el gradiente **se aproxima a cero** exponencialmente conforme nos alejamos en el tiempo. La red "olvida" eventos pasados y no puede aprender dependencias largas.

Exploding Gradient 📈

Si los pesos son mayores que 1, el gradiente **crece exponencialmente**, desestabilizando el entrenamiento con actualizaciones gigantescas. Se mitiga con **gradient clipping**: si la norma del gradiente supera un umbral, se reescala para limitar su magnitud.

⚠ El vanishing gradient es el motivo principal por el que las RNN simples fracasan con secuencias de más de 10–20 pasos. LSTM y GRU nacen precisamente para resolver este problema.

Las tres puertas de la LSTM

El núcleo de la LSTM es su **célula de estado** C_t, protegida y modulada por tres puertas aprendibles.



Puerta de olvido (*forget gate*)

Decide qué parte del estado anterior C_{t-1} se descarta. Genera valores entre 0 (olvidar todo) y 1 (recordar todo) mediante una sigmoide aplicada a $[h_{t-1}, x_t]$.



Puerta de entrada (*input gate*)

Controla qué nueva información se incorpora al estado. Combina una sigmoide (¿cuánto?) con una tangente hiperbólica (¿qué valor nuevo?) para actualizar C_t.



Puerta de salida (*output gate*)

Determina qué parte del estado de la célula se expone como estado oculto h_t. Filtra la información relevante para la predicción actual.

Gracias a las conexiones aditivas en C_t, el gradiente puede fluir sin degradarse durante muchos pasos de tiempo.

GRU: Gated Recurrent Unit

Propuesta por Cho et al. en 2014, la **GRU** simplifica la LSTM fusionando las puertas de olvido y entrada en una única **puerta de actualización** (*update gate*), y eliminando la célula de estado separada.

Con **menos parámetros**, la GRU es más rápida de entrenar y generaliza bien en conjuntos de datos de tamaño medio. En muchos benchmarks de series temporales su rendimiento es comparable al de la LSTM.

Puertas de la GRU

Update gate z_t

Decide cuánta información del pasado conservar frente a la nueva entrada. Reemplaza a las puertas de olvido y entrada de la LSTM.

Reset gate r_t

Controla cuánto del estado oculto pasado se usa para calcular el nuevo estado candidato. Permite olvidar contextos irrelevantes.

Cómo funciona la ventana deslizante

Dada una serie temporal $X = [x_1, x_2, ..., x_T]$, se define una ventana de tamaño **w**. Se desplaza por la serie generando pares:

- **Entrada:** $[x_t, x_{t+1}, ..., x_{t+w-1}]$ — los últimos w valores conocidos
- **Objetivo:** x_{t+w} — el valor a predecir (o varios, en predicción multistep)

El tamaño de la ventana es un **hiperparámetro crítico**: muy pequeño pierde contexto, muy grande añade ruido y coste computacional. Se ajusta mediante validación cruzada temporal.

Consideraciones prácticas

- Normalizar dentro de cada ventana (MinMax o Z-score) para estabilizar el entrenamiento
- Con datos diarios, ventanas de 7, 14 o 30 días suelen capturar patrones semanales o mensuales
- En predicción multistep, distinguir entre estrategia directa, recursiva y MIMO

Componentes de una Serie Temporal

Tendencia

Dirección general a largo plazo: creciente, decreciente o estacionaria. Puede ser lineal o no lineal. Se extrae mediante medias móviles o regresión.

Estacionalidad

Patrones periódicos y predecibles: ventas más altas en Navidad, mayor consumo eléctrico en verano. El período puede ser diario, semanal o anual.

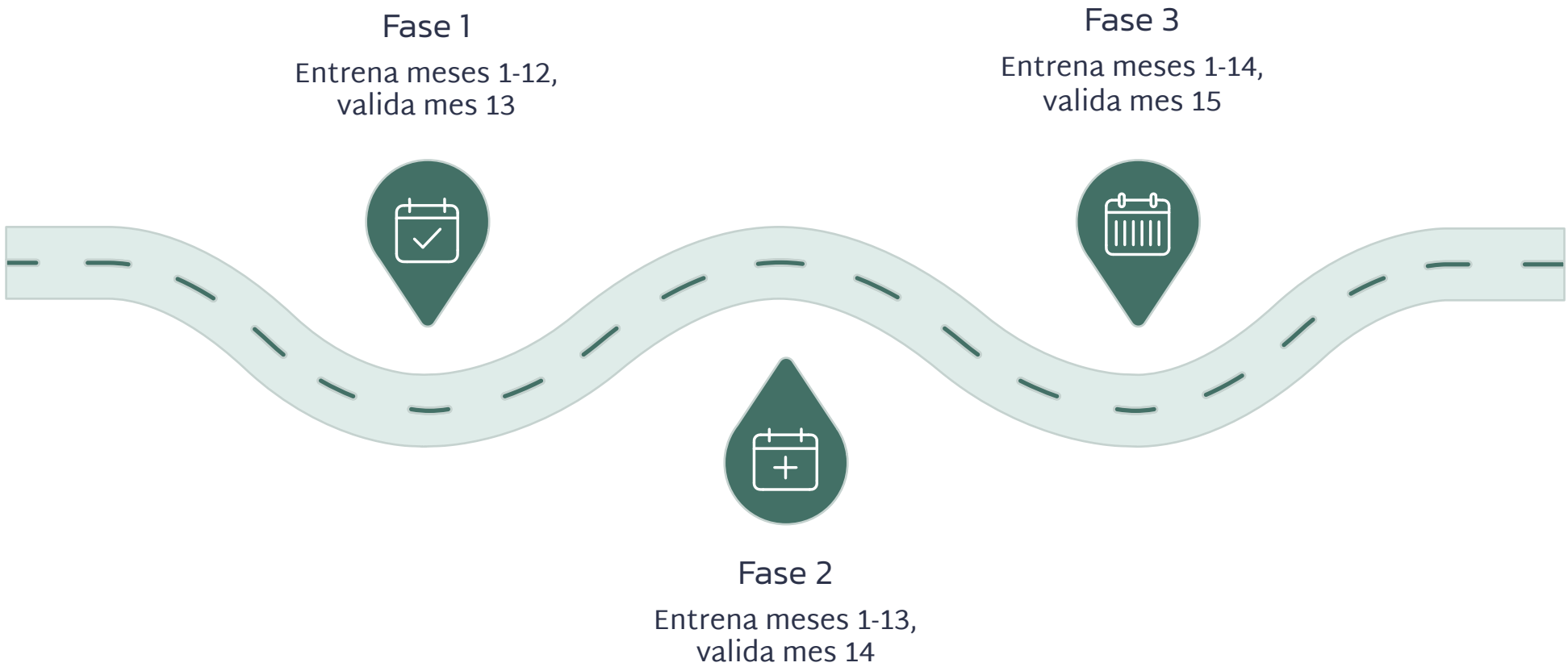
Residuo (ruido)

Lo que queda tras eliminar tendencia y estacionalidad. Idealmente es ruido blanco, sin estructura. Si tiene patrón, el modelo aún no captura toda la información.

Ciclo

Fluctuaciones no periódicas de larga duración, como ciclos económicos. A diferencia de la estacionalidad, su período no es fijo ni constante.

La descomposición clásica puede ser **aditiva** ($Y = T + S + R$) o **multiplicativa** ($Y = T \times S \times R$), según si los componentes se suman o se escalan entre sí.



En cada iteración del **walk-forward**, la ventana de entrenamiento se amplía con el período de validación anterior. Esto simula fielmente cómo se reentrenan los modelos en producción conforme llegan nuevos datos.

Modelos Clásicos vs. Redes Recurrentes

Criterio	ARIMA/SARIMA	Prophet	LSTM / GRU
Datos necesarios	Pocos (<500 obs.)	Pocos–medios	Muchos (>1000 obs.)
Interpretabilidad	Alta	Alta	Baja (caja negra)
Estacionalidad	SARIMA la modela explícitamente	Excelente soporte múltiple	Aprendida implícitamente
Relaciones no lineales	Limitada	Limitada	Muy alta
Series multivariadas	VAR (extensión)	No directamente	Nativa
Coste de entrenamiento	Muy bajo	Bajo	Alto (GPU recomendada)

Regla práctica: empieza siempre con un modelo clásico como baseline. Si las redes recurrentes no lo superan significativamente, el modelo simple gana por navaja de Occam.

S12 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S12_01_series_temporales — 45 min

Series temporales: *baselines*, descomposición y SARIMAX

sprints/S12/notebooks/S12_01_series_temporales.ipynb

CUADERNO

S12_02_redes_recurrentes — 45 min

RNN, LSTM y GRU frente al *baseline*

sprints/S12/notebooks/S12_02_redes_recurrentes.ipynb